



Урок 7 Аналитические функции

1. Аналитические функции
2. Три группы аналитических функций
3. Агрегирующие функции
4. Функции смещения
5. Функции ранжирования
6. Фреймы
7. Window Frame
8. Вопросы на собеседовании
9. Резюме

Аналитические функции

Аналитические функции SQL, также известные как оконные функции, играют ключевую роль в современном анализе данных. Они позволяют выполнять сложные вычисления на множестве строк, сохраняя при этом каждую строку в результирующем наборе данных. Эти функции расширяют возможности традиционных агрегатных функций, таких как `SUM`, `AVG`, и `COUNT`, позволяя выполнять операции над подмножествами данных. Благодаря своей мощности и гибкости аналитические функции стали незаменимым инструментом для

аналитиков и разработчиков, стремящихся к более глубокому пониманию данных и оптимизации производительности запросов.

Аналитическая (оконная) функция — это функция, выполняющая вычисления на основе определенного набора записей, которые являются окном (партицией). Оконная функция применяется к данным внутри окна и возвращает одиночное значение в качестве результата вычисления.

 Одно из главных преимуществ оконных функций заключается в том, что они возвращают ровно то же количество записей, которое получили на вход. Представьте, что вы хотите рассчитать некоторое значение для группы строк, объединённых общим признаком (например, id пользователя). Если бы вы воспользовались оператором GROUP BY, то на выходе вместо исходного количества строк в группе получили бы одну строку с результатом. При группировке так происходит всегда — число строк в результирующей таблице всегда равно количеству групп в исходной таблице. В то же время оконная функция позволяет проводить те же расчёты с агрегацией по группам, но при этом сохраняет структуру исходной таблицы: для каждой записи, принадлежащей определённой группе, в отдельном столбце просто указывается результат агрегации.

Аналитические (оконные) функции позволяют реализовать сложную логику, такую как расчет нарастающей суммы, определение дат первых и последних визитов клиента, и многое другое.

Данные делятся на **партиции (окна)** по определённому полю или полям. Внутри каждой партиции данные сортируются, после чего выполняется их анализ.

Пример

Рассмотрим пример. У нас есть пользователи, совершающие покупки. Для каждого пользователя в таблице фиксируется дата и время покупки, а также сумма покупки.

user_id	payment_sum	payment_dttm
1	5	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43
1	50	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22
3	2	2025-05-08 14:28:59
3	34	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02

Пример запроса с оконной функцией, вычисляющий номер покупки для каждого пользователя в хронологическом порядке, и сумму с нарастающим итогом всех покупок для каждого пользователя:

example.sql ×

```
1 SELECT user_id,  
2        payment_dttm,  
3        ROW_NUMBER() OVER  
4          (PARTITION BY user_id  
5            ORDER BY payment_dttm) AS num,  
6        SUM(payment_sum) OVER  
7          (PARTITION BY user_id  
8            ORDER BY payment_dttm) AS cume_sum  
9 FROM payments
```

Рассмотрим подробнее синтаксис оконной функции. Оконная функция используется с ключевым словом **OVER**. После **OVER** в скобках указывается описание окна. **PARTITION BY** определяет поля для разбиения данных на группы. **ORDER BY** задаёт порядок сортировки данных внутри каждой партии.

1 <функция>(<поле>) OVER (PARTITION BY <партия> ORDER BY <сортировка> <фрейм>)

- **функция** — оконная функция над выбранным полем;
- **партия** — поле или набор полей, по которым определяется группа;
 - в нашем случае разбиваем на группы по user_id

user_id	payment_sum	payment_dttm
1	5	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43
1	50	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22
3	2	2025-05-08 14:28:59
3	34	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02

- **сортировка** — поле или набор полей, по которым строки сортируются внутри партии;
 - в нашем примере сортируем по дате совершения покупки

user_id	payment_sum	payment_dttm
1	50	2025-05-06 15:58:02
1	12	2025-05-06 16:17:43
1	5	2025-05-06 18:00:34
2	33	2025-05-06 09:13:22
3	34	2025-05-05 11:39:05
3	2	2025-05-08 14:28:59
4	6	2025-05-09 14:18:02

- **фрейм** — набор строк внутри окна, которыми мы оперируем.

- необязательная часть запроса

Сначала производится разбиение на партии, определенные в инструкции **OVER**, при необходимости записи сортируются, а затем уже к каждой записи внутри партии применяется функция. **ROW_NUMBER()** возвращает номер строки внутри партии в поле **num**, а **SUM()** вычисляет кумулятивную сумму значений внутри партии в поле **cum_sum** (с каждой новой строкой наша сумма нарастает, таким образом получается сумма с нарастающим итогом).

user_id	payment_sum	payment_dttm	user_id	num	cume_sum	payment_dttm
1	50	2025-05-06 18:00:34	1	1	50	2025-05-06 18:00:34
1	12	2025-05-06 16:17:43	1	2	62	2025-05-06 16:17:43
1	5	2025-05-06 15:58:02	1	3	67	2025-05-06 15:58:02
2	33	2025-05-06 09:13:22	2	1	33	2025-05-06 09:13:22
3	34	2025-05-08 14:28:59	3	1	34	2025-05-08 14:28:59
3	2	2025-05-05 11:39:05	3	2	36	2025-05-05 11:39:05
4	6	2025-05-09 14:18:02	4	1	6	2025-05-09 14:18:02



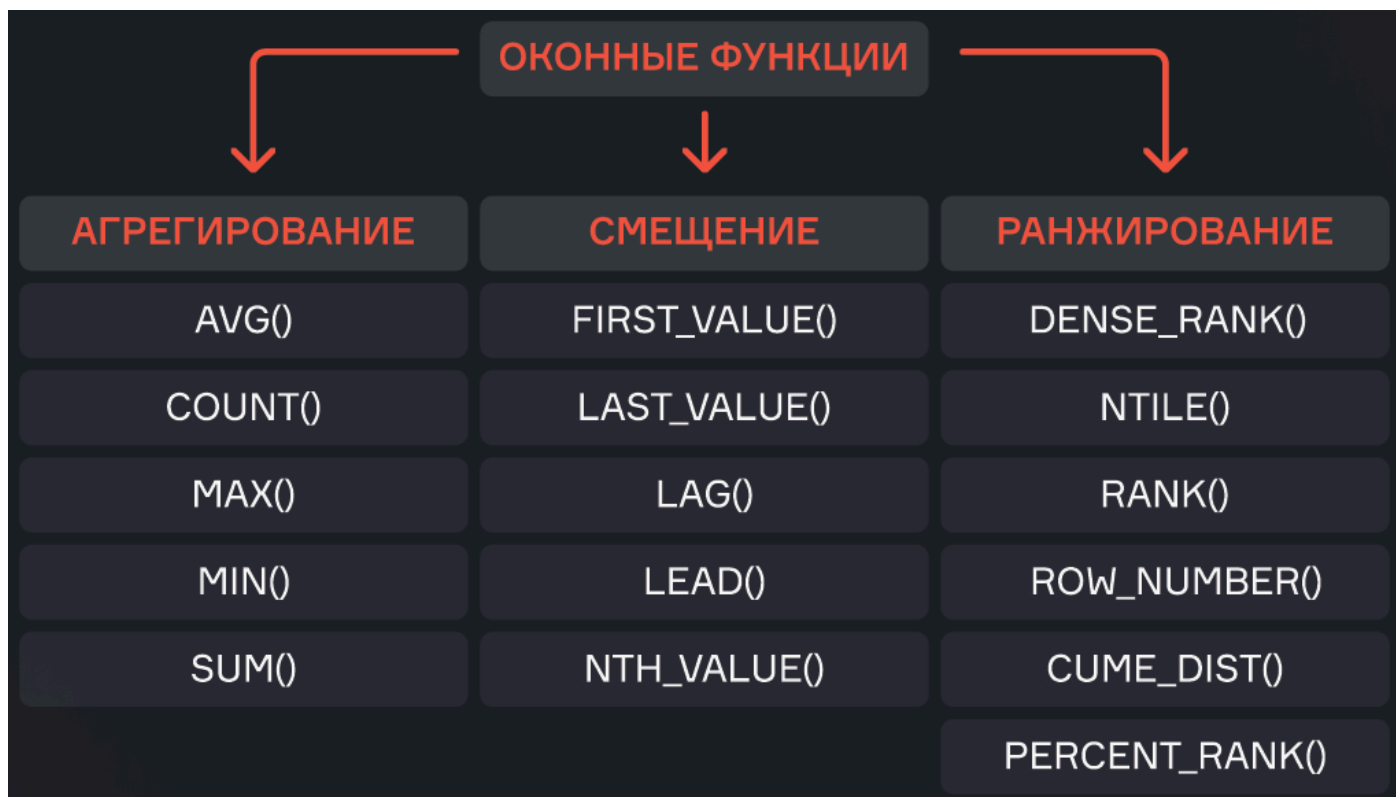
Почему же считается сумма именно текущей и всех предыдущих, а не вообще всех покупок пользователя? Дело в том, что при использовании оконных функций в паре с агрегирующими для каждой строки

определяется так называемая **рамка окна** — набор строк в её партии. Если в OVER указать ORDER BY, то по умолчанию рамка будет состоять из всех строк от начала партии до текущей строки (также в рамку будут включены строки, равные текущей строке по значению, указанному в ORDER BY). Именно поэтому в нашем примере сумма считается по каждому пользователю нарастающим итогом. Если же ORDER BY не указывать, то рамка по умолчанию будет состоять из всех строк партии, т.е. будет посчитана сумма всех покупок каждого пользователя. Также можно не указывать и PARTITION BY, тогда рамкой окна станет вся таблица, и мы просто посчитаем сумму покупок всех пользователей.

Три группы аналитических функций

Аналитические функции делятся на три основные группы:

1. Агрегирующие функции (SUM, AVG, MAX, MIN).
2. Функции смещения (LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE).
3. Функции ранжирования (ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUM_DIST, PERCENT_RANK).



Агрегирующие функции

Агрегирующие функции позволяют считать накопительные (кумулятивные) значения внутри окна:

- среднее (`AVG`)
- количество (`COUNT`)
- максимум (`MAX`)
- минимум (`MIN`)
- сумма (`SUM`)

Для рассмотрения оконных функций рассмотрим следующий запрос:

```
1 SELECT c.name,  
2        substr(ep.episode_id, 1, 3) AS season,  
3        COUNT(DISTINCT ep.episode_id) AS ep_cnt  
4 FROM characters AS c  
5 LEFT JOIN char_ep AS ce  
6     ON c.id = ce.character_id  
7 LEFT JOIN episodes AS ep  
8     ON ce.episode_id = ep.id  
9 GROUP BY 1, 2  
10 HAVING COUNT(ep.episode_id) > 5
```

В этом запросе мы объединили таблицу персонажей с таблицей эпизодов, получив на выходе всех персонажей и эпизоды, в которых они задействованы. Далее мы сгруппировали выборку по имени персонажа и по сезону, посчитав количество уникальных эпизодов, в которых наши персонажи участвовали, и ограничив выборку теми персонажами, которые участвовали в сезоне более пяти раз.

Результат запроса:

name	season	ep_cnt
Beth Smith	S01	11
Beth Smith	S02	8
Beth Smith	S03	10
Beth Smith	S04	10
Beth Smith	S05	9
Jerry Smith	S01	11
Jerry Smith	S02	9
Jerry Smith	S03	6
Jerry Smith	S04	8
Jerry Smith	S05	10
Morty Smith	S01	11
Morty Smith	S02	10

Обернув этот запрос в CTE, можно показать работу аналитических агрегирующих функций:

```

1 WITH char_in_episodes AS (
2     SELECT c.name,
3           substr(ep.episode_id, 1, 3) AS season,
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt
5     FROM characters AS c
6     LEFT JOIN char_ep AS ce
7           ON c.id = ce.character_id
8     LEFT JOIN episodes AS ep
9           ON ce.episode_id = ep.id
10    GROUP BY 1, 2
11    HAVING COUNT(ep.episode_id) > 5
12 )
13 SELECT name,
14        season,
15        ep_cnt,
16        SUM(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_sum,
17        AVG(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS ep_avg,
18        MIN(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_min,
19        MAX(ep_cnt) OVER (PARTITION BY name ORDER BY season)::int AS ep_max

```



```
20 FROM char_in_episodes;
```

name	season	ep_cnt	ep_sum	ep_avg	ep_min	ep_max
Beth Smith	S01	11	11	11.00	11	11
Beth Smith	S02	8	19	9.50	8	11
Beth Smith	S03	10	29	9.67	8	11
Beth Smith	S04	10	39	9.75	8	11
Beth Smith	S05	9	48	9.60	8	11
Jerry Smith	S01	11	11	11.00	11	11
Jerry Smith	S02	9	20	10.00	9	11
Jerry Smith	S03	6	26	8.67	6	11
Jerry Smith	S04	8	34	8.50	6	11
Jerry Smith	S05	10	44	8.80	6	11
Morty Smith	S01	11	11	11.00	11	11
Morty Smith	S02	10	21	10.50	10	11
Morty Smith	S03	10	31	10.33	10	11
Morty Smith	S04	10	41	10.25	10	11
Morty Smith	S05	10	51	10.20	10	11

Этот запрос показывает кумулятивные сумму (`ep_sum`), среднее (`ep_avg`), минимум (`ep_min`) и максимум (`ep_max`) по эпизодам для каждого персонажа по сезонам.

Функции смещения

Функции смещения позволяют обращаться к значениям соседних строк:

- `FIRST_VALUE` — первая строка в партииции
- `LAST_VALUE` — последняя строка в партииции
- `LAG` — значение в предыдущей строке партииции
- `LEAD` — значение в следующей строке партииции

- **NTH_VALUE** — значение в строке партии под номером N (переданным в качестве второго аргумента функции)

Пример работы функций смещения:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3           substr(ep.episode_id, 1, 3) AS season,  
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7     ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9     ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14        season,  
15        ep_cnt,  
16        FIRST_VALUE(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS first,  
17        LAST_VALUE(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS last,  
18        LAG(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS  
19        prev_season_cnt,  
20        LEAD(ep_cnt) OVER (PARTITION BY name ORDER BY season) AS  
21        next_season_cnt,  
22        NTH_VALUE(ep_cnt, 2) OVER (PARTITION BY name ORDER BY season) AS  
23        second_season_cnt  
24 FROM char_in_episodes;
```

name	season	ep_cnt	first	last	prev_season_cnt	next_season_cnt	second_season_cnt
Beth Smith	S01	11	11	11		8	
Beth Smith	S02	8	11	8	11	10	8
Beth Smith	S03	10	11	10	8	10	8
Beth Smith	S04	10	11	10	10	9	8
Beth Smith	S05	9	11	9	10		8
Jerry Smith	S01	11	11	11		9	
Jerry Smith	S02	9	11	9	11	6	9
Jerry Smith	S03	6	11	6	9	8	9
Jerry Smith	S04	8	11	8	6	10	9

В этом примере используются функции `FIRST_VALUE`, `LAST_VALUE`, `LAG`, `LEAD` и `NTH_VALUE` для получения первого (`first`), последнего (`last`), предыдущего (`prev_season_cnt`), следующего (`next_season_cnt`) и второго (`second_season_cn`) значений в рамке каждого окна.

Функции ранжирования

Функции ранжирования назначают порядковые номера строкам или ранжируют их внутри партиций:

- `ROW_NUMBER` — возвращает номер строки внутри партиции
- `RANK` — вычисляет ранг записи в рамках партиции, отбрасывая следующий ранг при повторяющихся значениях
- `DENSE_RANK` — вычисляет ранг записи в рамках партиции без пропусков при повторяющихся значениях
- `NTILE` — производит разбиение окна на заданное количество групп, которое передается в функцию в качестве аргумента, и возвращает номер группы

- **CUME_DIST** — возвращает относительный номер строки (кумулятивное распределение RANK/COUNT)
- **PERCENT_RANK** — возвращает относительный ранг $(RANK-1)/(COUNT-1)$

Пример работы функций ранжирования:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3           substr(ep.episode_id, 1, 3) AS season,  
4           COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7     ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9     ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14        season,  
15        ep_cnt,  
16        ROW_NUMBER() OVER (PARTITION BY name ORDER BY ep_cnt) AS rn,  
17        RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS rank,  
18        DENSE_RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS dense_rank,  
19        NTILE(2) OVER (PARTITION BY name ORDER BY ep_cnt) AS group_num,  
20        CUME_DIST() OVER (PARTITION BY name ORDER BY ep_cnt) AS cume_dist,  
21        PERCENT_RANK() OVER (PARTITION BY name ORDER BY ep_cnt) AS percent_rank  
22 FROM char_in_episodes;
```

name	season	ep_cnt	rn	rank	dense_rank	group_num	cume_dist	percent_rank
Beth Smith	S02	8	1	1	1	1	0.20	0.00
Beth Smith	S05	9	2	2	2	1	0.40	0.25
Beth Smith	S03	10	3	3	3	1	0.80	0.50
Beth Smith	S04	10	4	3	3	2	0.80	0.50
Beth Smith	S01	11	5	5	4	2	1.00	1.00
Jerry Smith	S03	6	1	1	1	1	0.20	0.00
Jerry Smith	S04	8	2	2	2	1	0.40	0.25
Jerry Smith	S02	9	3	3	3	1	0.60	0.50
Jerry Smith	S05	10	4	4	4	2	0.80	0.75
Jerry Smith	S01	11	5	5	5	2	1.00	1.00
Morty Smith	S03	10	1	1	1	1	0.80	0.00
Morty Smith	S02	10	2	1	1	1	0.80	0.00
Morty Smith	S04	10	3	1	1	1	0.80	0.00
Morty Smith	S05	10	4	1	1	2	0.80	0.00

Этот запрос показывает различия между `ROW_NUMBER` (столбец `rn`), `RANK`, `DENSE_RANK`, `NTILE` (столбец `group_num`), `CUME_DIST` и `PERCENT_RANK`:

- Функция `ROW_NUMBER` пронумеровала строки в рамках каждого персонажа. А переданное в качестве аргумента в функцию `NTILE` число 2 позволило разделить строки внутри каждой партии на две примерно равные части (влияет нечетность количества строк внутри партий) и пронумеровать каждую строку в соответствии со своей группой.
- Мы видим, что в третьем и в четвертом сезоне Бэт Смит играла по 10 раз и обе строки пронумерованы цифрой 3 как в `RANK`, так и в `DENSE_RANK`. Но следующая строка в `DENSE_RANK` будет иметь номер 4, а в `RANK` номер 5.
- Функция `CUME_DIST` показывает долю строк, которые меньше либо равны текущей строке. А функция `PERCENT_RANK` показывает долю строк, которые меньше текущей строки. Соответственно для последнего значения обеих функций это всегда единица. Для первого значения в `PERCENT_RANK` нет ни одной строки, значение которых меньше ее, а для кумулятивного распределения `CUME_DIST` первое значение — ненулевое.

Фреймы

Фреймы позволяют детализировать диапазон строк для агрегирования.

Существует два способа задания фреймов:

1. **ROWS** – работает со строками внутри партиции.
2. **RANGE** – работает с диапазоном значений внутри партиции.

ROWS

Указывает нижнюю и верхнюю границу окна по номерам строк.

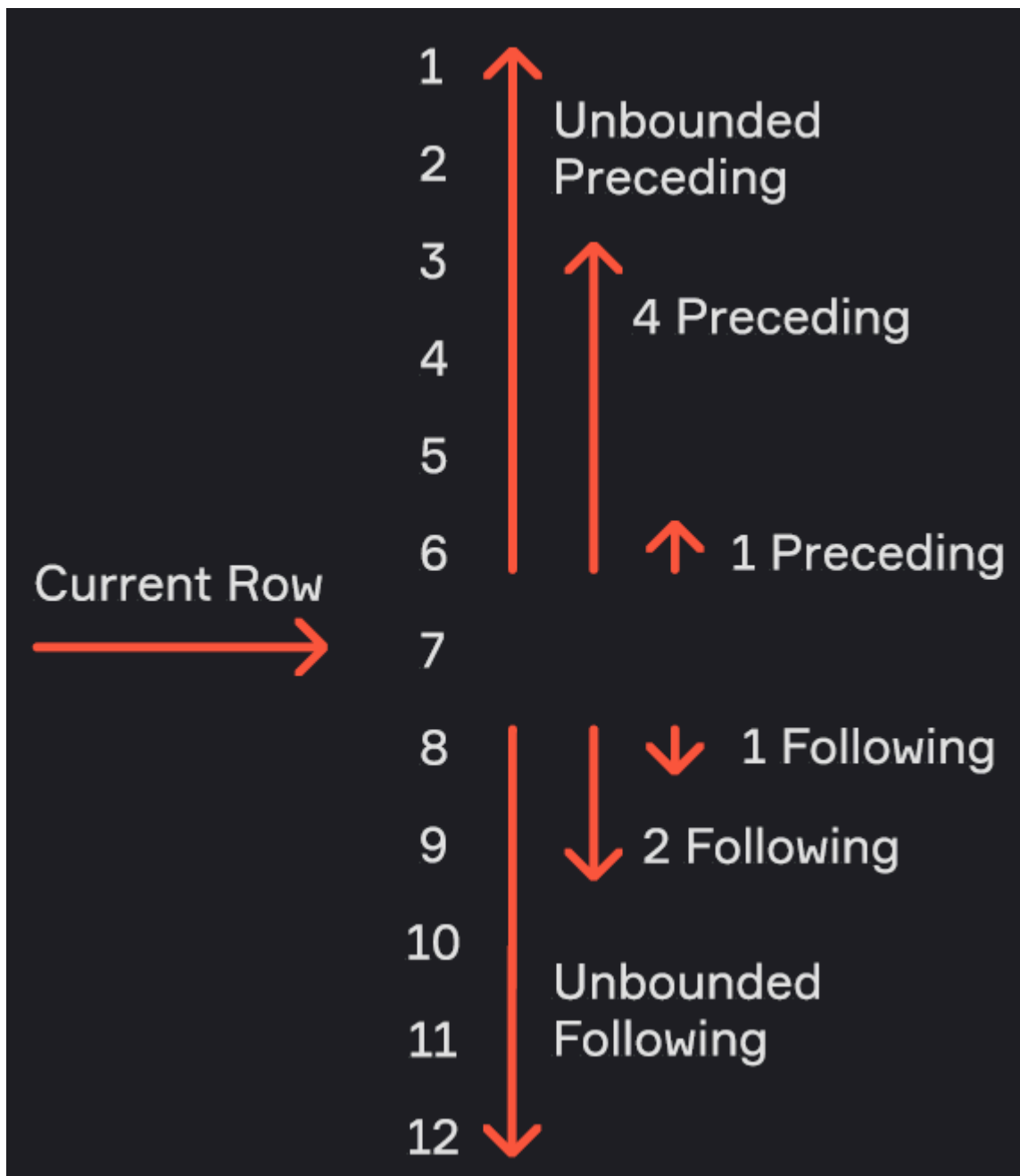
Синтаксис:

ROWS BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первой строки группы
- **UNBOUNDED FOLLOWING** – окно заканчивается на последней строке группы
- **CURRENT ROW** – окно начинается или заканчивается на текущей строке

Также можно указать номера строк:

- **<число строк> PRECEDING** – определяет число строк перед текущей строкой
- **<число строк> FOLLOWING** — определяет число строк после текущей строки



Например, у нас есть партиция из 12 строк. **ROWS** позволяет нам выбрать те строки, значения которых нам нужно агрегировать. Например, текущая строка номер 7, по умолчанию мы агрегируем для нее все строки, начиная с первой и до нее включительно. Но можно выбрать и другие строки, которые мы будем агрегировать. Например, начиная с третьей строки до текущей. Начиная с одной строки назад до текущей. Начиная с текущей до последней строки в окне. Начиная с текущей и еще две строки. Начиная с текущей и еще одну строку. Для этого мы используем нотацию **ROWS BETWEEN**, затем указываем нижнюю границу, **AND** и верхнюю границу.

Пример запроса с использованием фрейма ROWS:

```
1 WITH char_in_episodes AS (
```

```

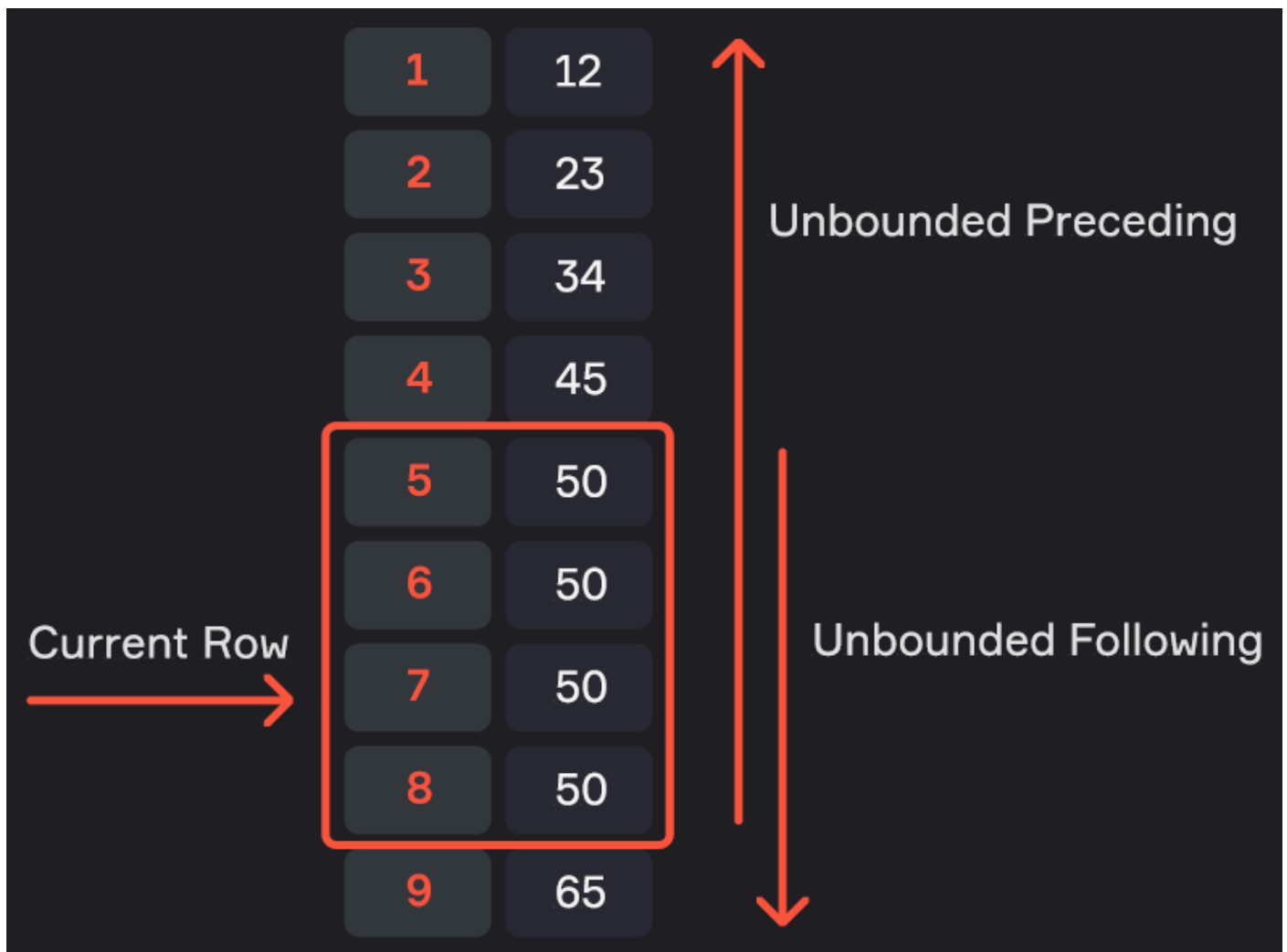
2  SELECT c.name,
3         substr(ep.episode_id, 1, 3) AS season,
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt
5  FROM characters AS c
6  LEFT JOIN char_ep AS ce
7         ON c.id = ce.character_id
8  LEFT JOIN episodes AS ep
9         ON ce.episode_id = ep.id
10 GROUP BY 1, 2
11 HAVING COUNT(ep.episode_id) > 5
12 )
13 SELECT name,
14        season,
15        ep_cnt,
16        SUM(ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN UNBOUNDED
17        PRECEDING AND CURRENT ROW)::int AS v1,
18        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN CURRENT
19        ROW AND UNBOUNDED FOLLOWING) AS v2,
20        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN 3
21        PRECEDING AND CURRENT ROW)::int AS v3,
22        SUM (ep_cnt) OVER (PARTITION BY name ORDER BY season ROWS BETWEEN CURRENT
23        ROW AND 2 FOLLOWING)::int AS v4
24 FROM char_in_episodes

```

name	season	ep_cnt	v1	v2	v3	v4
Beth Smith	S01	11	11	48.00	11	29
Beth Smith	S02	8	19	37.00	19	28
Beth Smith	S03	10	29	29.00	29	29
Beth Smith	S04	10	39	19.00	39	19
Beth Smith	S05	9	48	9.00	37	9
Jerry Smith	S01	11	11	44.00	11	26
Jerry Smith	S02	9	20	33.00	20	23
Jerry Smith	S03	6	26	24.00	26	24
Jerry Smith	S04	8	34	18.00	34	18
Jerry Smith	S05	10	44	10.00	33	10

RANGE

Указывает нижнюю и верхнюю границу окна по значениям строк.



RANGE BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первого значения группы
- **UNBOUNDED FOLLOWING** – окно заканчивается на последнем значении группы
- **CURRENT ROW** – окно начинается или заканчивается на текущем значении



По умолчанию, если мы не указали фрейм, применяется **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** : все записи, начиная с первой строки окна до текущей строки.

Всё, что до текущей строки
и само значение текущей строки

**RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW**

Текущая строка и всё, что после неё

**RANGE BETWEEN CURRENT ROW
AND UNBOUNDED FOLLOWING**

Значение по умолчанию

Пример

Все, что до текущего диапазона и само текущее значение:

```
1 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

Текущий диапазон и все, что после него:

```
1 RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
```

Пример запроса с использованием фрейма RANGE:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3         substr(ep.episode_id, 1, 3) AS season,  
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7         ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9         ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )
```

```

13 SELECT name,
14     season,
15     ep_cnt,
16     SUM(ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN
    UNBOUNDED PRECEDING AND CURRENT ROW)::int AS v1,
17     SUM(ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN
    UNBOUNDED PRECEDING AND CURRENT ROW)::int AS v2,
18     SUM (ep_cnt) OVER (PARTITION BY name ORDER BY ep_cnt RANGE BETWEEN CURRENT
    ROW AND UNBOUNDED FOLLOWING) AS v3
19 FROM char_in_episodes

```

name	season	ep_cnt	v1	v2	v3
Beth Smith	S02	8	8	8	48.00
Beth Smith	S05	9	17	17	40.00
Beth Smith	S03	10	37	37	31.00
Beth Smith	S04	10	37	37	31.00
Beth Smith	S01	11	48	48	11.00
Jerry Smith	S03	6	6	6	44.00
Jerry Smith	S04	8	14	14	38.00
Jerry Smith	S02	9	23	23	30.00
Jerry Smith	S05	10	33	33	21.00
Jerry Smith	S01	11	44	44	11.00

Window Frame

Window Frame — именованная партиция, которую можно использовать в запросе больше одного раза. Window Frame улучшает читаемость запроса и в некоторых СУБД увеличивает скорость запросов.

Синтаксис Window Frame:

```

1 **WINDOW <alias>** AS (PARTITION BY <партиция> ORDER BY <сортировка> <фрейм>)

```

Определение именованной партиции производится после запроса.

Пример:

```
1 WITH char_in_episodes AS (  
2     SELECT c.name,  
3         substr(ep.episode_id, 1, 3) AS season,  
4         COUNT(DISTINCT ep.episode_id) AS ep_cnt  
5     FROM characters AS c  
6     LEFT JOIN char_ep AS ce  
7         ON c.id = ce.character_id  
8     LEFT JOIN episodes AS ep  
9         ON ce.episode_id = ep.id  
10    GROUP BY 1, 2  
11    HAVING COUNT(ep.episode_id) > 5  
12 )  
13 SELECT name,  
14     season,  
15     ep_cnt,  
16     SUM(ep_cnt) OVER w::int AS an_sum,  
17     AVG(ep_cnt) OVER w AS an_avg,  
18     MIN(ep_cnt) OVER w::int AS an_min,  
19     MAX(ep_cnt) OVER w::int AS an_max  
20 FROM char_in_episodes  
21 WINDOW w AS (PARTITION BY name ORDER BY season)
```

name	season	ep_cnt	an_sum	an_avg	an_min	an_max
Beth Smith	S01	11	11	11.00	11	11
Beth Smith	S02	8	19	9.50	8	11
Beth Smith	S03	10	29	9.67	8	11
Beth Smith	S04	10	39	9.75	8	11
Beth Smith	S05	9	48	9.60	8	11
Jerry Smith	S01	11	11	11.00	11	11
Jerry Smith	S02	9	20	10.00	9	11
Jerry Smith	S03	6	26	8.67	6	11
Jerry Smith	S04	8	34	8.50	6	11
Jerry Smith	S05	10	44	8.80	6	11
Morty Smith	S01	11	11	11.00	11	11
Morty Smith	S02	10	21	10.50	10	11
Morty Smith	S03	10	31	10.33	10	11
Morty Smith	S04	10	41	10.25	10	11
Morty Smith	S05	10	51	10.20	10	11

Вопросы на собеседовании

- **Как посчитать кумулятивную сумму?**

С помощью аналитической функции `SUM`. Мы делим на партии наши данные, сортируем их и применяем к нему агрегирующую функцию.

- **Что будет, если в оконных функциях не указать фрейм?**

Мы агрегируем данные, начиная с самой первой строки окна по текущую включительно.

- **Какие варианты указания фрейма вы знаете?**

- по строкам (`ROWS`)
- по значениям строк (`RANGE`).

- **Чем отличается RANK и DENSE_RANK?**

RANK при ранжировании делает пропуски, если значение нескольких строк одинаково. А DENSE_RANK после одинаковых строк присваивает последующее значение без пропусков.

Резюме

- **Аналитическая (оконная) функция** — это функция, выполняющая вычисления на основе определенного набора записей, которые являются окном (партицией). Оконная функция применяется к отсортированным данным и возвращает одиночное значение в качестве результата вычисления.
- Одно из главных преимуществ оконных функций заключается в том, что они возвращают ровно то количество записей, которое получили на вход.
- Синтаксис оконной функции: **<функция>(<поле>) OVER (PARTITION BY <партиция> ORDER BY <сортировка> <фрейм>)**, где:
 - **функция** — оконная функция над выбранным полем;
 - **партиция** — поле или набор полей, по которым определяется группа;
 - **сортировка** — поле или набор полей, по которым строки отсортировываются внутри партии;
 - **фрейм** — набор строк внутри окна, которыми мы оперируем.
- Аналитические функции делятся на три основные группы:
 - Агрегирующие функции (SUM, AVG, MAX, MIN) позволяют считать накопительные (кумулятивные) значения внутри окна.
 - Функции смещения (LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE) позволяют обращаться к значениям соседних строк.
 - Функции ранжирования (ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUM_DIST, PERCENT_RANK) назначают порядковые номера строкам или ранжируют их внутри партий.
- Фреймы позволяют детализировать диапазон строк для агрегирования. Существует два способа задания фреймов:
 - **ROWS** — работает со строками внутри партии;
 - **RANGE** — работает с диапазоном значений внутри партии.
- Синтаксис задания фрейма:

ROWS BETWEEN <нижняя граница> AND <верхняя граница>

- **UNBOUNDED PRECEDING** — окно начинается с первой строки группы
- **UNBOUNDED FOLLOWING** — окно заканчивается на последней строке группы
- **CURRENT ROW** — окно начинается или заканчивается на текущей строке
- **<число строк> PRECEDING** — определяет число строк перед текущей строкой
- **<число строк> FOLLOWING** — определяет число строк после текущей строки

Для **RANGE** указание числа строк не актуально.

- **Window Frame** — именованная партиция, которую можно использовать в запросе больше одного раза. Window Frame улучшает читаемость запроса и в некоторых СУБД увеличивает скорость запросов.

Синтаксис Window Frame:

WINDOW <alias> AS (PARTITION BY <партиция> ORDER BY <сортировка> <фрейм>)